



Universidade
de Fortaleza

Aspectos Teóricos da Computação — Projeto AV3

NeyScript: Construção de um Compilador Front-end

Análise Léxica (JFlex), Sintática e Semântica (CUP) e Integração

Uma linguagem de programação temática (Neymar Jr.) com estrutura convencional: tipos, funções, condicionais, laços e expressões.

João Vitor dos Santos Silva

Matrícula 2210358

JFlex 1.9.1 · CUP 11b · Java (JDK 11+)
Fortaleza – Ceará
2026

Sumário

1. Introdução.....	3
2. Objetivos.....	3
3. Ferramentas e arquitetura do projeto.....	3
4. Especificação da linguagem NeyScript.....	4
5. Analisador Léxico com JFlex (src/lexer.flex).....	6
6. Analisador Sintático e Semântico com CUP (src/parser.cup).....	7
7. Integração (src/Main.java).....	9
8. Demonstração e testes.....	9
9. Como compilar e executar (README).....	11
10. Mapa de entregáveis.....	12
11. Resultados e conclusão.....	12
12. Repositório e apresentação em vídeo.....	12

1. Introdução

O front-end de um compilador é a parte responsável por ler e compreender o código-fonte, antes de qualquer geração de código de máquina. Ele é organizado em três fases encadeadas: a análise léxica, que agrupa os caracteres em unidades mínimas com significado (os tokens); a análise sintática, que verifica se a sequência de tokens obedece à gramática da linguagem; e a análise semântica, que verifica se o programa faz sentido (tipos compatíveis, identificadores declarados, retornos coerentes, entre outros).

Este trabalho tem como objetivo a construção de um analisador léxico utilizando JFlex, de um analisador sintático e semântico utilizando CUP (JCCup), e a integração de ambos em uma ferramenta única, capaz de processar um arquivo de código-fonte como entrada e produzir, de ponta a ponta, o fluxo de tokens, a validação gramatical e os diagnósticos semânticos.

A linguagem criada — NeyScript — substitui as palavras-chave clássicas por termos do universo do Neymar Jr., preservando integralmente a estrutura de uma linguagem convencional. Os tipos primitivos viram estatísticas (camisa10, ousadia, canto, semGol); as funções viram partidas; a função principal é o próprio neymar; condicionais são expressas por seJogar e banco; a repetição por replay e rodada; a impressão na tela é a narração (narra); e as chamadas de função usam o prefixo assistencia, simbolizando o passe que aciona a jogada. Toda a implementação foi feita em Java, com JFlex 1.9.1 para a fase léxica e CUP 11b para as fases sintática e semântica.

2. Objetivos

2.1. Objetivo geral

Construir o front-end completo de um compilador para a linguagem NeyScript, integrando um analisador léxico (JFlex) a um analisador sintático e semântico (CUP), de modo que o programa processe um código-fonte como entrada e produza tokens, verificações gramaticais e checagens de sentido até uma saída esperada.

2.2. Objetivos específicos

1. Implementar, com JFlex, um analisador léxico (Scanner) que reconheça palavras-chave, identificadores, constantes inteiras e de ponto flutuante, operadores, delimitadores e comentários de linha e de bloco.
2. Implementar, com CUP, um analisador sintático (Parser) que descreva a estrutura do programa, a declaração de variáveis, as estruturas de controle (condicional e laços), a definição e a chamada de funções, além de expressões e atribuições.
3. Embutir a análise semântica nas ações da gramática, com tabela de símbolos por escopos, verificação de declaração antes do uso, compatibilidade de tipos, aridade e tipo dos argumentos e coerência de retorno.
4. Integrar as especificações .flex e .cup, fazendo a saída do Scanner servir de entrada ao Parser, e demonstrar a solução completa sobre um arquivo input.txt.
5. Fornecer scripts de build e execução e um arquivo README explicando como compilar e executar o Scanner, o Parser e a solução integrada.

3. Ferramentas e arquitetura do projeto

As ferramentas adotadas são declarativas: a equipe descreve as regras em arquivos de especificação e as ferramentas convertem essas regras em código Java. O JFlex lê src/lexer.flex e gera o Scanner

(Lexer.java); o CUP lê src/parser.cup e gera o Parser (parser.java) junto com a tabela de constantes de tokens (sym.java); por fim, o javac compila tudo, incluindo o driver de integração src/Main.java.

```
src/lexer.flex --[ JFlex ]--> gen/Lexer.java      (Scanner / léxico)
src/parser.cup --[ CUP   ]--> gen/parser.java + sym.java (Parser / sint.+sem.)
src/Main.java  --[ javac ]--> build/Main.class      (integração / pipeline)
```

Figura 1 – Fluxo de geração: das especificações ao código Java executável.

A estrutura de diretórios do repositório é a seguinte:

```
neyscript/
├── lib/                                # dependencias (jars)
│   ├── jflex-full-1.9.1.jar
│   ├── java-cup-11b.jar
│   └── java-cup-11b-runtime.jar
├── src/
│   ├── lexer.flex                    # especificacao JFlex (lexico)
│   ├── parser.cup                    # especificacao CUP (sintatico + semantico)
│   └── Main.java                     # driver de integracao (pipeline)
├── gen/                              # gerados: Lexer.java, parser.java, sym.java
├── build/                            # .class compilados
├── input.txt                         # programa-exemplo valido
├── input_erros.txt                   # programa com erros (demonstracao)
├── build.sh                          # gera (JFlex+CUP) e compila (javac)
├── run.sh                           # executa o compilador
└── README.md
```

4. Especificação da linguagem NeyScript

4.1. Tabela de tokens

A tabela a seguir lista as palavras-chave reconhecidas pelo Scanner, seu equivalente em linguagens convencionais e a justificativa temática. Cada palavra-chave é reconhecida antes do padrão de identificador, garantindo que um identificador não “absorva” uma keyword.

Token	Lexema	Equivalente	Justificativa temática
NEYMAR	neymar	main	Personagem central do programa, como a função principal
CAMISA10	camisa10	int	Tipo inteiro; o número clássico dos craques
OUSADIA	ousadia	float	Tipo real; valores de precisão, habilidade ou estatística
CANTO	canto	string	Tipo texto; o canto da torcida (cadeias de caracteres)
SEM_GOL	semGol	void	Função sem retorno, ou seja, sem “gol” produzido
PARTIDA	partida	definição de função	Uma função vira uma “partida” executável
NARRA	narra	print	O narrador anuncia/imprime um valor na tela
SE_JOGAR	seJogar	if	Condição: “se jogar”, executa o bloco
BANCO	banco	else	Caminho alternativo: se não entrou em campo,

Token	Lexema	Equivalente	Justificativa temática
			ficou no banco
REPLAY	replay	while	Repetição: revê a jogada enquanto a condição vale
RODADA	rodada	for	Laço com contador, como as rodadas de um campeonato
GOL	gol	return	Resultado final devolvido pela partida
ASSISTENCIA	assistencia	chamada de função	O passe que aciona outra jogada/função
APITO	apito	break	Encerra o laço, como o apito final
DRIBLAR	driblar	continue	Pula para a próxima iteração, driblando o resto
CAMPEAO	campeao	true (1)	Literal booleano verdadeiro
VICE	vice	false (0)	Literal booleano falso
LESIONADO	lesionado	null	Ausência de valor; atribuível a qualquer tipo
ID	gols, jogos	identificador	Nomes definidos pelo programador
NUM_INT	10, 2, 7	inteiro	Literais inteiros
NUM_FLOAT	9.5, 10.0	real	Literais decimais
LIT_TEXTO	"Neymar Jr"	string literal	Literais de texto entre aspas

Tabela 1 – Tokens da linguagem NeyScript e seus equivalentes convencionais.

4.2. Operadores, delimitadores e comentários

Categoria	Símbolos
Aritméticos	+ - * /
Relacionais	== != < > <= >=
Atribuição	=
Delimitadores	; , () { }
Comentário de linha	// até o fim da linha
Comentário de bloco	/* qualquer coisa, multi-linha */

Tabela 2 – Operadores, delimitadores e comentários auxiliares.

4.3. Exemplo mínimo

```
partida camisa10 soma(camisa10 a, camisa10 b) {
    gol a + b;
}

neymar() {
    canto nome = "Neymar Jr";
    camisa10 total = assistencia soma(118, 136);
    narra(nome);
    seJogar (total >= 250) {
        narra("LENDÁ!");
    } banco {
        narra("ainda nao");
    }
}
```

5. Analisador Léxico com JFlex (src/lexer.flex)

A especificação JFlex define macros (expressões regulares nomeadas), as regras de reconhecimento e as ações em Java executadas a cada casamento. O Scanner lê o texto caractere a caractere e devolve, a cada chamada de `next_token()`, o próximo token com sua linha e coluna. As diretivas `%line` e `%column` rastreiam a posição; `%cup` integra o Scanner ao runtime do CUP; e cada palavra-chave aparece antes do padrão `{Identifier}` para respeitar a regra de prioridade do JFlex.

5.1. Macros e literais

```
Digit      = [0-9]
Letter     = [a-zA-Z_]
Identifier = {Letter}{Letter}{Digit}*
IntegerLiteral = {Digit}+
FloatLiteral = {Digit}+ "." {Digit}+
StringLiteral = \"([^\\"\\n]|\\.)*\"
CommentSingle = \"/\" [^\r\n]*
CommentMulti  = \"/*\" ~\"*/\"
Comment       = {CommentSingle} | {CommentMulti}
```

Trecho 1 – Macros de identificadores, constantes, strings e comentários.

Os literais já são convertidos para os tipos Java correspondentes no momento do reconhecimento: inteiros viram `Integer`, decimais viram `Double` e cadeias de texto têm as aspas removidas antes de virarem `String`.

5.2. Palavras-chave, operadores e ações

```
<YYINITIAL> {
  /* palavras-chave (antes de {Identifier}) */
  "camisa10" { return symbol(sym.CAMISA10); } /* int */
  "seJogar"  { return symbol(sym.SE_JOGAR); } /* if */
  "banco"    { return symbol(sym.BANCO); }   /* else */
  "replay"   { return symbol(sym.REPLAY); }  /* while */
  "gol"      { return symbol(sym.GOL); }     /* return */

  /* literais ja convertidos para tipos Java */
  {FloatLiteral} { return symbol(sym.NUM_FLOAT, Double.parseDouble(yytext())); }
  {IntegerLiteral} { return symbol(sym.NUM_INT, Integer.parseInt(yytext())); }
  {StringLiteral} { String s = yytext();
                      return symbol(sym.LIT_TEXT0, s.substring(1, s.length()-1)); }

  "==" { return symbol(sym.EQ); }  "!=" { return symbol(sym.NEQ); }
  "+" { return symbol(sym.PLUS); } ";" { return symbol(sym.SEMI); }

  {Identifier} { return symbol(sym.ID, yytext()); }
  {Comment}    { /* descarta */ }
  {WhiteSpace} { /* descarta */ }
}
[^] { throw new RuntimeException("ERRO LEXICO: caractere invalido ..."); }
```

Trecho 2 – Reconhecimento de keywords, literais, operadores e identificadores.

Comentários e espaços em branco são descartados; qualquer caractere fora do alfabeto da linguagem dispara um erro léxico com a posição exata. A classe gerada chama-se `Lexer`

(gen/Lexer.java) e expõe o método `next_token()`, consumido pelo Parser. Com isso, a fase léxica cobre todos os itens exigidos: palavras-chave, identificadores, constantes inteiras e de ponto flutuante, operadores, delimitadores e comentários de linha e de bloco.

6. Analisador Sintático e Semântico com CUP (src/parser.cup)

6.1. Terminais, não-terminais e precedência

Os terminais correspondem aos tokens devolvidos pelo Scanner; alguns carregam valor (ID é String, NUM_INT é Integer, NUM_FLOAT é Double, LIT_TEXTO é String). Os não-terminais `expr` e `tipo` são tipados como String porque carregam, durante a análise, o tipo resultante usado pelas checagens semânticas. A precedência segue o estilo de C: atribuição (à direita), depois igualdade, relacionais, adição/subtração e, com maior precedência, multiplicação/divisão.

```
terminal          NEYMAR, CAMISA10, OUSADIA, CANTO, SEM_GOL;
terminal          PARTIDA, SE_JOGAR, BANCO, REPLAY, RODADA, GOL, ASSISTENCIA, NARRA;
terminal          CAMPEAO, VICE, LESIONADO, APITO, DRIBLAR;
terminal          PLUS, MINUS, MULT, DIV, EQ, NEQ, LT, GT, LEQ, GEQ, ASSIGN;
terminal          SEMI, COMMA, LPAREN, RPAREN, LBRACE, RBRACE;
terminal String    ID, LIT_TEXTO;    terminal Integer NUM_INT;    terminal Double NUM_FLOAT;

non terminal String tipo, expr;    /* carregam o tipo durante a analise */

precedence right ASSIGN;
precedence left  EQ, NEQ;
precedence left  LT, GT, LEQ, GEQ;
precedence left  PLUS, MINUS;
precedence left  MULT, DIV;
```

Trecho 3 – Declaração de terminais, não-terminais tipados e precedência.

6.2. Gramática (resumo)

A gramática descreve a estrutura do programa, a declaração de variáveis, as estruturas de controle, a definição e a chamada de funções, além de expressões e atribuições — exatamente os itens exigidos no enunciado.

```
programa          ::= lista_decl
lista_decl        ::= lista_decl decl | decl
decl              ::= decl_func | decl_neymar
decl_func         ::= PARTIDA tipo ID LPAREN lista_params RPAREN bloco
decl_neymar       ::= NEYMAR LPAREN RPAREN bloco
tipo              ::= CAMISA10 | OUSADIA | CANTO | SEM_GOL
params            ::= params COMMA param | param          param ::= tipo ID
bloco             ::= LBRACE lista_stmt RBRACE
stmt              ::= stmt_decl_var | stmt_atrib | stmt_se_jogar | stmt_replay
                  | stmt_rodada | stmt_retorno | stmt_assist | stmt_narra
                  | stmt_apito | stmt_driblar | bloco
stmt_decl_var     ::= tipo ID SEMI | tipo ID ASSIGN expr SEMI
stmt_atrib        ::= ID ASSIGN expr SEMI
stmt_se_jogar     ::= SE_JOGAR LPAREN expr RPAREN bloco banco_opt
stmt_replay       ::= REPLAY LPAREN expr RPAREN bloco
stmt_rodada       ::= RODADA LPAREN tipo ID ASSIGN expr SEMI expr SEMI
                  ID ASSIGN expr RPAREN bloco
stmt_retorno      ::= GOL expr SEMI | GOL SEMI
stmt_assist       ::= ASSISTENCIA ID LPAREN lista_args RPAREN SEMI
```

```

expr      ::= expr op expr | MINUS expr | LPAREN expr RPAREN
           | ASSISTENCIA ID LPAREN lista_args RPAREN | ID
           | NUM_INT | NUM_FLOAT | LIT_TEXTO | CAMPEAO | VICE | LESIONADO

```

Trecho 4 – Resumo das produções da gramática NeyScript.

6.3. Análise semântica embutida

O bloco parser code {: ... :} contém toda a infraestrutura semântica, executada dentro das ações de cada produção:

- **TabelaSimbolos** — pilha de escopos (Deque<Map<String, Simbolo>>), com abrirEscopo/fecharEscopo, declarar e buscar.
- **Simbolo** — guarda nome, tipo, se é partida, a lista de tipos dos parâmetros e a linha de declaração.
- **unificarTipos(t1, t2, op, linha)** — decide o tipo resultante de uma operação; relacionais retornam camisa10 (0/1, estilo C) e + concatena canto.
- **compativelAtribuicao(destino, origem)** — permite a coerção camisa10 → ousadia e o valor lesionado (null) em qualquer tipo; rejeita os demais.
- **checarArgs(simbolo, args, linha)** — valida a aridade e o tipo de cada argumento das chamadas com assistencia.
- **tipoRetornoAtual / paramsAtual** — pilhas que guardam o tipo de retorno e os parâmetros da partida corrente, alimentando as checagens de gol e de chamada.
- **loopDepth** — contador que garante que apito (break) e driblar (continue) só apareçam dentro de laços.

Exemplo de ação semântica na declaração de variável com inicialização:

```

tipo:t ID:nome ASSIGN expr:e SEMI
{:
  if (t.equals("semGol"))
    parser.erroSemantico("variavel " + nome + " nao pode ter tipo semGol", tleft);
  if (!parser.tabela.declarar(new Simbolo(nome, t, false, null, tleft)))
    parser.erroSemantico("variavel " + nome + " ja declarada no escopo atual",
tleft);
  if (!parser.compativelAtribuicao(t, e))
    parser.erroSemantico("atribuicao incompativel: " + nome + " (" + t + ") = " + e,
tleft);
:}

```

Trecho 5 – Ação semântica: declaração, redeclaração e compatibilidade de tipos.

O CUP gera gen/parser.java (o Parser LALR(1)) e gen/sym.java (as constantes inteiras de cada token, usadas tanto pelo Scanner quanto pelo driver). Ao compilar, o CUP reporta zero conflitos, evidenciando uma gramática sem ambiguidades.

6.4. Checagens semânticas implementadas

- Tabela de símbolos com escopos aninhados (global, por partida, por bloco de rodada).
- Declaração antes do uso, para variáveis e para partidas.
- Detecção de redeclaração no mesmo escopo.
- Compatibilidade de tipos em atribuições, com coerção camisa10 → ousadia e aceitação de lesionado (null).
- Operadores aritméticos exigem operandos numéricos; o operador + concatena canto (string).
- Operadores relacionais retornam camisa10 (0/1, estilo C).

- Condições de seJogar, replay e rodada devem ser numéricas.
- assistencia confere existência, categoria (é partida?), aridade e tipo de cada argumento.
- Retorno (gol) compatível com o tipo declarado da partida; partidas semGol não retornam valor nem aparecem em expressões.
- apito (break) e driblar (continue) só são válidos dentro de laços.
- narra (print) aceita qualquer valor, exceto semGol.

7. Integração (src/Main.java)

O driver Main liga o Scanner ao Parser e organiza a execução em três etapas. A integração segue o modelo pull: o Parser solicita o próximo token ao Scanner (next_token()) somente quando precisa, e não há acoplamento manual entre códigos de token e nomes — o mapeamento código → nome é feito por reflexão sobre a classe sym.

```
Lexer lex = new Lexer(new StringReader(fonte)); // 1. Scanner (JFlex)
parser p = new parser(lex); // 2. liga ao Parser (CUP)
p.parse(); // 3. token -> sintaxe -> semantica
p.tabela.imprimir(); // 4. tabela de simbolos
```

Trecho 6 – Núcleo da integração: uma chamada (p.parse()) dispara todo o pipeline.

As três etapas executadas pelo driver são:

1. Etapa 1 — Léxica: instancia o Lexer, percorre next_token() até o EOF e imprime a tabela linha · coluna · classe · lexema.
2. Etapa 2 — Sintática + Semântica: passa um novo Lexer ao parser.parse(); os erros são acumulados em parser.mensagensErro sem abortar a análise.
3. Etapa 3 — Relatório: imprime a tabela de símbolos e o veredito final: GOOOOOL!! (aceito) ou CARTAO VERMELHO (com a lista de erros).

Os scripts automatizam o ciclo completo:

```
./build.sh # JFlex -> CUP -> javac (gera gen/ e build/)
./run.sh # executa o pipeline completo sobre input.txt
./run.sh --lex input.txt # apenas a analise lexica (stream de tokens)
./run.sh --parse input.txt # lex + sintatica + semantica (padrao)
./run.sh input_erros.txt # demonstracao dos diagnosticos semanticos
```

8. Demonstração e testes

8.1. Compilação do projeto (build)

A saída real de ./build.sh confirma que o JFlex gera o Scanner, o CUP gera o Parser e o javac compila tudo, sem erros e sem conflitos — atestando uma gramática LALR(1) sem ambiguidade.

```
$ ./build.sh
[1/3] JFlex ..... gerando Lexer.java
[2/3] CUP ..... gerando parser.java + sym.java
0 errors and 0 warnings
41 terminals, 30 non-terminals, and 73 productions declared,
producing 145 unique parse states.
0 conflicts detected (0 expected).
```

```
[3/3] javac ..... compilando classes
OK - build concluido em build/
```

Trecho 7 – Build: 41 terminais, 30 não-terminais, 73 produções, 145 estados, 0 conflitos.

8.2. Programa válido (input.txt)

O arquivo input.txt exercita todos os elementos da linguagem: três partidas (com retorno inteiro, real e semGol), o ponto de entrada neymar(), parâmetros tipados, declarações com e sem inicializador, chamadas com assistencia (aridade e tipos conferidos), impressão com narra, condicional seJogar/banco, laços replay e rodada com apito e driblar, operadores aritméticos e relacionais, literais inteiros, reais e de texto, concatenação de canto, coerção camisa10 → ousadia, o valor lesionado (null), os literais campeão/vice e comentários de linha e de bloco.

```
$ ./run.sh input.txt
ETAPA 1 - ANALISE LEXICA (Scanner JFlex)
LINHA COL TOKEN LEXEMA/VALOR
14 1 PARTIDA
14 18 ID somarGols
41 27 NUM_INT 118
75 19 LIT_TEXTO OLE
...
total de tokens: 348

ETAPA 2 - ANALISE SINTATICA + SEMANTICA (CUP)
>> programa analisado com sucesso.

ETAPA 3 - RELATORIO FINAL
G00000L!! programa aceito sem erros lex/sint/sem.
```

Trecho 8 – Execução sobre o programa válido: 348 tokens, programa aceito.

A tabela de símbolos do escopo global, impressa ao final, registra as quatro partidas declaradas com seu tipo, categoria, linha e a assinatura de parâmetros:

Nome	Tipo	Categoria	Linha	Parâmetros
somarGols	camisa10	partida	14	[camisa10, camisa10]
mediaGols	ousadia	partida	20	[camisa10, camisa10]
anunciar	semGol	partida	31	[canto]
neymar	camisa10	partida	38	[]

Tabela 3 – Tabela de símbolos (escopo global) gerada para input.txt.

8.3. Programa com erros (input_errores.txt)

O segundo arquivo de teste concentra erros propositais para demonstrar o diagnóstico. O compilador não para no primeiro erro: percorre todo o programa, acumula as mensagens e reporta cada uma com a sua linha. Foram detectados 13 erros semânticos, todos com a localização precisa.

#	Linha	Diagnóstico emitido
1	9	partida 'dobro' já declarada
2	14	partida 'semGol' não pode retornar valor
3	19	variável 'x' já declarada no escopo atual
4	22	atribuição incompatível: 'y' (camisa10) = ousadia

#	Linha	Diagnóstico emitido
5	24	variável 'z' não declarada
6	26	variável 'w' não pode ter tipo semGol
7	29	operação '+' inválida entre tipos canto e camisa10
8	31	partida 'dobro' espera 1 argumento(s), recebeu 3
9	34	argumento 1 de 'dobro': esperado camisa10, recebeu canto
10	36	partida 'inexistente' não declarada
11	38	'x' não é partida
12	40	'apito' fora de loop (replay/rodada)
13	41	'driblar' fora de loop (replay/rodada)

Tabela 4 – Os 13 diagnósticos semânticos detectados em input_errores.txt.

```
$ ./run.sh input_errores.txt
ETAPA 3 - RELATORIO FINAL
CARTAO VERMELHO - 13 erro(s) encontrado(s):
- ERRO SEMANTICO (linha 9): partida 'dobro' ja declarada
- ERRO SEMANTICO (linha 22): atribuicao incompativel: 'y' (camisa10) = ousadia
- ERRO SEMANTICO (linha 31): partida 'dobro' espera 1 argumento(s), recebeu 3
- ERRO SEMANTICO (linha 40): 'apito' fora de loop (replay/rodada)
... (13 diagnosticos no total)
```

Trecho 9 – Veredito final do programa com erros: CARTÃO VERMELHO.

8.4. Resumo dos casos de teste

Caso de teste	Entrada	Resultado obtido
Compilação	./build.sh	0 erros · 0 conflitos · classes geradas
Programa válido	input.txt	348 tokens · 0 erros · GOOOOOL!!
Programa com erros	input_errores.txt	13 erros detectados · CARTÃO VERMELHO
Análise léxica isolada	./run.sh --lex	348 tokens listados (linha-coluna-classe-lexema)

Tabela 5 – Todos os casos retornaram o resultado esperado: o front-end opera de ponta a ponta.

9. Como compilar e executar (README)

Requisitos: JDK 11 ou superior (testado em OpenJDK 21/23) e bash. Os jars do JFlex e do CUP já acompanham o projeto na pasta lib/ e não precisam ser instalados. A partir da raiz do repositório:

```
# 1) compila o front-end (JFlex -> CUP -> javac)
./build.sh

# 2) executa o pipeline completo sobre input.txt
./run.sh

# variantes
./run.sh input.txt           # qualquer arquivo de entrada
./run.sh --lex input.txt     # apenas a analise lexica
./run.sh --parse input.txt   # lex + sintatica + semantica (padrao)
./run.sh input_errores.txt   # demonstracao dos diagnosticos semanticos
```

10. Mapa de entregáveis

A tabela abaixo relaciona cada item solicitado no enunciado ao artefato correspondente do projeto.

Item solicitado	Artefato entregue
Especificação JFlex (.flex) + classe Scanner gerada	src/lexer.flex → gen/Lexer.java
Especificação CUP (.cup) + classes Parser e Sym geradas	src/parser.cup → gen/parser.java, gen/sym.java
Scripts de teste e demonstração da integração	build.sh, run.sh, src/Main.java
Código-fonte de entrada	input.txt (e input_erros.txt)
README de compilação e execução	README.md
Apresentação em vídeo	Link do YouTube (Seção 12)

Tabela 6 – Correspondência entre os requisitos do trabalho e os artefatos entregues.

11. Resultados e conclusão

A implementação validou, na prática, a construção de um front-end de compilador completo e integrado. A fase léxica, descrita em JFlex, reconhece corretamente palavras-chave, identificadores, constantes inteiras e reais, operadores, delimitadores e comentários, descartando ruído e sinalizando caracteres inválidos com posição. A fase sintática, descrita em CUP, expressa a estrutura do programa, declarações, controle de fluxo, funções, expressões e atribuições com uma gramática LALR(1) sem conflitos.

A análise semântica, embutida nas ações da gramática, mostrou-se robusta: a tabela de símbolos por escopos, a verificação de declaração antes do uso, a compatibilidade de tipos com coerção, a conferência de aridade e tipos de argumentos e a coerência de retorno foram exercitadas tanto pelo programa válido quanto pelo programa com erros. No primeiro, o compilador gerou 348 tokens e aceitou o programa; no segundo, detectou os 13 erros propositais sem interromper a análise, reportando cada um com a sua linha.

A integração, conduzida pelo driver Main.java em modelo pull, confirmou que a saída do Scanner alimenta diretamente o Parser e que uma única chamada dispara todo o pipeline, do texto ao relatório final. Conclui-se que os objetivos foram plenamente atingidos: o projeto entrega um compilador front-end funcional de ponta a ponta, com especificações .flex e .cup, classes geradas, scripts de build e execução, arquivos de entrada de demonstração e documentação, proporcionando uma compreensão prática das três fases iniciais da compilação.

12. Repositório e apresentação em vídeo

Repositório (código-fonte completo): github.com/joaoviitorsx/neyscript

Apresentação em vídeo (YouTube): <https://youtu.be/ISp4sNQTS6I>

O vídeo apresenta a visão geral das três fases, a explicação de lexer.flex, parser.cup e do driver Main.java, e as provas de execução: a compilação (./build.sh), o programa válido (input.txt, aceito) e o programa com erros (input_erros.txt, com os diagnósticos semânticos).